20. 데이터 저장소 및 데이터베이스 구현 분석

개요

Lighthouse는 Ethereum 2.0 합의 클라이언트로서 대량의 블록체인 데이터를 효율적으로 저장하고 검색해야 합니다. 이 문서는 Lighthouse의 데이터 저장소 시스템인 `store` 모듈의 구현을 상세히 분석하며, LevelDB와 ReDB를 활용한 데이터베이스 아키텍처, Hot-Cold 저장 전략, 그리고 다양한 데이터 구조와 최적화 기법들을 실제 코드와 함께 설명합니다.

저장소 아키텍처 개요

전체 구조

Lighthouse의 저장소 시스템은 다음과 같은 계층적 구조로 설계되어 있습니다:

```
┌─────────────────────────────────────────────────────────────┐
| Store Module |
├─────────────────────────────────────────────────────────────┤
| ┌──────────────┐ ┌──────────────────────────┐ |
| | HotColdDB | | MemoryStore | |
| | | | | |
| | - Hot Storage | | - In-Memory HashMap | |
| | - Cold Storage | | - Testing Purpose | |
| | - Blob Storage | | - Fast Access | |
| └──────────────┘ └──────────────────────────┘ |
```

```
├─────────────────────────────────────────────────────────────┤
| Database Backend Layer |
| |
| ┌──────────────┐ ┌──────────────────────────┐ |
| | LevelDB | | ReDB | |
| | | | | |
| | - LSM Tree | | - LMDB-like | |
| | - Compression | | - ACID Transactions | |
| | - Compaction | | - Memory Mapped | |
| └──────────────┘ └──────────────────────────┘ |
└─────────────────────────────────────────────────────────────┘
```

핵심 컴포넌트

1. Store 인터페이스 계층

`store/src/lib.rs` 에서 정의된 핵심 트레이트들:

```rust
// 키-값 저장소의 기본 인터페이스
pub trait KeyValueStore<E: EthSpec>: Sync + Send + Sized + 'static {
/// 바이트 데이터 조회
fn get_bytes(&self, column: DBColumn, key: &[u8]) -> Result<Option<Vec<u8>>, Er
/// 바이트 데이터 저장
fn put_bytes(&self, column: DBColumn, key: &[u8], value: &[u8]) -> Result<(), E
/// 동기화된 바이트 데이터 저장 (fsync 포함)
```

```
fn put_bytes_sync(&self, column: DBColumn, key: &[u8], value: &[u8]) -> Result<

/// 디스크 동기화

fn sync(&self) -> Result<(), Error>;

/// 키 존재 여부 확인

fn key_exists(&self, column: DBColumn, key: &[u8]) -> Result<bool, Error>;

/// 키 삭제

fn key_delete(&self, column: DBColumn, key: &[u8]) -> Result<(), Error>;

/// 원자적 배치 작업 실행

fn do_atomically(&self, batch: Vec<KeyValueStoreOp>) -> Result<(), Error>;

/// 컬럼 압축

fn compact_column(&self, column: DBColumn) -> Result<(), Error>;

/// 컬럼 반복자

fn iter_column_from<K: Key>(&self, column: DBColumn, from: &[u8]) -> ColumnIter

}



// 고수준 아이템 저장소 인터페이스

pub trait ItemStore<E: EthSpec>: KeyValueStore<E> + Sync + Send + Sized + 'stat

/// 구조화된 아이템 저장

fn put<I: StoreItem>(&self, key: &Hash256, item: &I) -> Result<(), Error> {

let column = I::db_column();

let key = key.as_slice();

self.put_bytes(column, key, &item.as_store_bytes())

}
```

```
/// 구조화된 아이템 조회
fn get<I: StoreItem>(&self, key: &Hash256) -> Result<Option<I>, Error> {
let column = I::db_column();
let key = key.as_slice();
match self.get_bytes(column, key)? {
Some(bytes) => Ok(Some(I::from_store_bytes(&bytes[..])?)),
None => Ok(None),
}
}
}
```

2. 데이터베이스 컬럼 시스템

Lighthouse는 데이터를 논리적으로 분리하기 위해 컬럼 기반 저장 방식을 사용합니다:

```
#[derive(Debug, Clone, Copy, PartialEq, IntoStaticStr, EnumString, EnumIter)]
pub enum DBColumn {
/// 데이터베이스 메타데이터
#[strum(serialize = "bma")]
BeaconMeta,
/// 블록 데이터 (핫 DB: SSZ 인코딩된 블라인드 블록, 콜드 DB: 8바이트 슬롯)
#[strum(serialize = "blk")]
BeaconBlock,
/// 블롭 사이드카 데이터
```

```
#[strum(serialize = "blb")]
BeaconBlob,
/// 데이터 컬럼 사이드카
#[strum(serialize = "bdc")]
BeaconDataColumn,
/// 핫 DB의 전체 BeaconState (완료된 또는 포크 경계 상태)
#[strum(serialize = "ste")]
BeaconState,
/// 핫 DB의 압축된 BeaconStateDiff
#[strum(serialize = "hsd")]
BeaconStateHotDiff,
/// 핫 DB의 비콘 상태 스냅샷
#[strum(serialize = "hsn")]
BeaconStateHotSnapshot,
/// 실행 페이로드 (최종화 체크포인트 이후 블록용)
#[strum(serialize = "exp")]
ExecPayload,
/// 포크 선택 데이터
#[strum(serialize = "frk")]
ForkChoice,
/// 공개키 캐시
#[strum(serialize = "pkc")]
PubkeyCache,
```

```
// ... 기타 컬럼들
}

impl DBColumn {
/// 대부분의 데이터베이스 키는 32바이트이지만, 일부 프리저 DB 키는 8바이트
pub fn key_size(self) -> usize {
match self {
Self::BeaconBlock | Self::BeaconState | Self::BeaconBlob => 32,
Self::BeaconBlockRoots | Self::BeaconStateRoots => 8,
Self::BeaconDataColumn => 40, // 32 + 8 (블록 루트 + 컬럼 인덱스)
// ... 기타 케이스들
}
}
}
```

Hot-Cold 저장 전략

HotColdDB 구조

Lighthouse의 핵심 저장소인 `HotColdDB` 는 효율적인 데이터 관리를 위해 Hot-Cold 분할 전략을 사용합니다:

```
#[derive(Debug)]
pub struct HotColdDB<E: EthSpec, Hot: ItemStore<E>, Cold: ItemStore<E>> {
```

```
/// 핫-콜드 DB 분할 지점 (슬롯과 상태 루트)
pub(crate) split: RwLock<Split>,
/// 데이터베이스에 저장된 블록 및 상태 범위의 시작 슬롯
anchor_info: RwLock<AnchorInfo>,
/// 블롭 저장 범위의 시작 슬롯
blob_info: RwLock<BlobInfo>,
/// 데이터 컬럼 저장 범위의 시작 슬롯
data_column_info: RwLock<DataColumnInfo>,
/// 저장소 설정
pub(crate) config: StoreConfig,
/// 계층적 차이 구조 설정
pub  hierarchy: HierarchyModuli,
/// 압축된 과거 데이터를 포함하는 콜드 데이터베이스
pub  cold_db: Cold,
/// 블롭 데이터베이스 (None이면 cold_db 사용)
pub  blobs_db: Cold,
/// 중복되지만 빠른 접근이 가능한 최근 데이터를 포함하는 핫 데이터베이스
pub  hot_db: Hot,
/// 역직렬화된 블록과 블롭의 LRU 캐시
block_cache: Mutex<BlockCache<E>>,
/// 비콘 상태 캐시
pub  state_cache: Mutex<StateCache<E>>,
/// 과거 상태 및 계층적 차이 버퍼 캐시
```

```
historic_state_cache: Mutex<HistoricStateCache<E>>,

/// 체인 스펙

pub spec: Arc<ChainSpec>,

_phantom: PhantomData<E>,

}
```

분할 전략

Hot-Cold 분할은 다음 원칙에 따라 작동합니다:

1. **Hot Storage**: `split.slot` 이상의 슬롯을 가진 최근 상태들

- 빠른 접근이 필요한 데이터
- 포크 선택과 검증에 사용되는 데이터
- 전체 상태와 차이(diff) 데이터 모두 저장

2. **Cold Storage**: `split.slot` 미만의 슬롯을 가진 과거 상태들

- 압축된 형태로 저장
- 복원 지점(restore point) 기반 저장
- 계층적 차이(hierarchical diff) 사용

데이터베이스 백엔드 구현

백엔드 추상화

`database/interface.rs` 에서 정의된 백엔드 추상화:

```
pub enum BeaconNodeBackend<E: EthSpec> {

#[cfg(feature = "leveldb")]
```

```
LevelDb(leveldb_impl::LevelDB<E>),
#[cfg(feature = "redb")]
Redb(redb_impl::Redb<E>),
}


impl<E: EthSpec> BeaconNodeBackend<E> {
pub fn open(config: &StoreConfig, path: &Path) -> Result<Self, Error> {
match config.backend {
#[cfg(feature = "leveldb")]
DatabaseBackend::LevelDb => {
leveldb_impl::LevelDB::open(path).map(BeaconNodeBackend::LevelDb)
}
#[cfg(feature = "redb")]
DatabaseBackend::Redb => {
redb_impl::Redb::open(path).map(BeaconNodeBackend::Redb)
}
}
}
}
```

LevelDB 구현

`database/leveldb_impl.rs` 의 LevelDB 구현:

```
pub struct LevelDB<E: EthSpec> {
db: Database<BytesKey>,
_phantom: PhantomData<E>,
}


impl<E: EthSpec> LevelDB<E> {
pub fn open(path: &Path) -> Result<Self, Error> {
let mut options = Options::new();
options.create_if_missing = true;
let db = Database::open(path, options)?;
Ok(Self {
db,
_phantom: PhantomData,
})
}
pub fn put_bytes_with_options(
&self,
col: DBColumn,
key: &[u8],
val: &[u8],
opts: WriteOptions,
) -> Result<(), Error> {
```

```
let  column_key = get_key_for_col(col, key);

// 메트릭 업데이트

metrics::inc_counter_vec(&metrics::DISK_DB_WRITE_COUNT, &[col.into()]);

metrics::inc_counter_vec_by(

&metrics::DISK_DB_WRITE_BYTES,

&[col.into()],

val.len() as u64,

);

let  timer = metrics::start_timer(&metrics::DISK_DB_WRITE_TIMES);

self.db

.put(opts.into(), BytesKey::from_vec(column_key), val)

.map_err(Into::into)

.map(|()| {

metrics::stop_timer(timer);

})

}

pub  fn  get_bytes(&self, col: DBColumn, key: &[u8]) -> Result<Option<Vec<u8>>,

let  column_key = get_key_for_col(col, key);

metrics::inc_counter_vec(&metrics::DISK_DB_READ_COUNT, &[col.into()]);

let  timer = metrics::start_timer(&metrics::DISK_DB_READ_TIMES);

self.db

.get(self.read_options(), BytesKey::from_vec(column_key))

.map_err(Into::into)
```

```
.map(|opt| {
opt.inspect(|bytes| {
metrics::inc_counter_vec_by(
&metrics::DISK_DB_READ_BYTES,
&[col.into()],
bytes.len() as u64,
);
metrics::stop_timer(timer);
})
})
}
}
```

키 구성 전략

데이터베이스 키는 컬럼 접두사와 실제 키를 결합하여 구성됩니다:

```
pub fn get_key_for_col(column: DBColumn, key: &[u8]) -> Vec<u8> {
let mut result = column.as_bytes().to_vec(); // 3바이트 컬럼 접두사
result.extend_from_slice(key); // 실제 키 데이터
result
}

// 예시: BeaconBlock 컬럼의 키 구성
```

```
// "blk" + [32바이트 블록 해시] = 35바이트 키
```

저장소 설정 및 최적화

StoreConfig 구조

`config.rs` 에서 정의된 저장소 설정:

```rust
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct StoreConfig {
/// 인메모리 블록 캐시에 저장할 최대 블록 수
pub block_cache_size: NonZeroUsize,
/// 인메모리 상태 캐시에 저장할 최대 상태 수
pub state_cache_size: NonZeroUsize,
/// 상태 캐시 가득 참 시 제거할 최소 상태 수
pub state_cache_headroom: NonZeroUsize,
/// 블록, 상태 차이 및 기타 압축 값의 압축 레벨
pub compression_level: i32,
/// 인메모리 과거 상태 캐시에 저장할 최대 과거 상태 수
pub historic_state_cache_size: NonZeroUsize,
/// 메모리에 저장할 최대 콜드 HDiffBuffer 수
pub cold_hdiff_buffer_cache_size: NonZeroUsize,
/// 메모리에 저장할 최대 핫 HDiffBuffer 수
pub hot_hdiff_buffer_cache_size: NonZeroUsize,
```

```rust
/// 초기화 시 데이터베이스 압축 여부
pub compact_on_init: bool,
/// 데이터베이스 정리 중 압축 여부
pub compact_on_prune: bool,
/// 초기화 및 최종화 시 페이로드 정리 여부
pub prune_payloads: bool,
/// 사용할 데이터베이스 백엔드
pub backend: DatabaseBackend,
/// 상태 차이 계층 구조
pub hierarchy_config: HierarchyConfig,
/// 블롭 데이터 가용성 경계보다 오래된 블롭 정리 여부
pub prune_blobs: bool,
/// 에포크 단위 블롭 정리 빈도 (기본값: 1)
pub epochs_per_blob_prune: u64,
/// 에포크 단위 블롭 정리 마진 (기본값: 0)
pub blob_prune_margin_epochs: u64,
}

impl Default for StoreConfig {
fn default() -> Self {
Self {
block_cache_size: new_non_zero_usize(64),
state_cache_size: new_non_zero_usize(128),
```

```
state_cache_headroom: new_non_zero_usize(1),
compression_level: 1,
historic_state_cache_size: new_non_zero_usize(1),
cold_hdiff_buffer_cache_size: new_non_zero_usize(16),
hot_hdiff_buffer_cache_size: new_non_zero_usize(1),
compact_on_init: false,
compact_on_prune: true,
prune_payloads: true,
backend: DEFAULT_BACKEND,
hierarchy_config: HierarchyConfig::default(),
prune_blobs: true,
epochs_per_blob_prune: 1,
blob_prune_margin_epochs: 0,
}
}
}
```

압축 및 성능 최적화

```
impl StoreConfig {
/// 현재 압축 레벨에서 len 바이트의 압축 후 크기 추정
pub fn estimate_compressed_size(&self, len: usize) -> usize {
if self.compression_level == 0 {
```

```
len
} else {
len / 2  // 대략적인 압축 비율
}
}
/// 바이트 데이터 압축
pub fn compress_bytes(&self, ssz_bytes: &[u8]) -> Result<Vec<u8>, Error> {
let mut compressed_value =
Vec::with_capacity(self.estimate_compressed_size(ssz_bytes.len()));
let mut encoder = Encoder::new(&mut compressed_value, self.compression_level)
.map_err(Error::Compression)?;
encoder.write_all(ssz_bytes).map_err(Error::Compression)?;
encoder.finish().map_err(Error::Compression)?;
Ok(compressed_value)
}
}
```

메모리 저장소 구현

MemoryStore 구조

테스트 및 개발 목적으로 사용되는 인메모리 저장소:

```
type DBMap = BTreeMap<BytesKey, Vec<u8>>;
```

```
pub struct MemoryStore<E: EthSpec> {
db: RwLock<DBMap>,
_phantom: PhantomData<E>,
}

impl<E: EthSpec> MemoryStore<E> {
pub fn open() -> Self {
Self {
db: RwLock::new(BTreeMap::new()),
_phantom: PhantomData,
}
}
}

impl<E: EthSpec> KeyValueStore<E> for MemoryStore<E> {
fn get_bytes(&self, col: DBColumn, key: &[u8]) -> Result<Option<Vec<u8>>, Error
let column_key = BytesKey::from_vec(get_key_for_col(col, key));
Ok(self.db.read().get(&column_key).cloned())
}
fn put_bytes(&self, col: DBColumn, key: &[u8], val: &[u8]) -> Result<(), Error>
let column_key = BytesKey::from_vec(get_key_for_col(col, key));
self.db.write().insert(column_key, val.to_vec());
```

```
Ok(())
}
fn do_atomically(&self, batch: Vec<KeyValueStoreOp>) -> Result<(), Error> {
for op in batch {
match op {
KeyValueStoreOp::PutKeyValue(col, key, value) => {
let column_key = get_key_for_col(col, &key);
self.db.write().insert(BytesKey::from_vec(column_key), value);
}
KeyValueStoreOp::DeleteKey(col, key) => {
let column_key = get_key_for_col(col, &key);
self.db.write().remove(&BytesKey::from_vec(column_key));
}
}
}
Ok(())
}
}
```

데이터 직렬화 및 저장

StoreItem 트레이트

모든 저장 가능한 데이터 타입이 구현해야 하는 트레이트:

```rust
pub trait StoreItem: Sized {
/// 이 아이템이 저장될 컬럼 식별
fn db_column() -> DBColumn;
/// self를 바이트로 직렬화
fn as_store_bytes(&self) -> Vec<u8>;
/// 바이트에서 self를 역직렬화
fn from_store_bytes(bytes: &[u8]) -> Result<Self, Error>;
/// 키-값 저장소 연산으로 변환
fn as_kv_store_op(&self, key: Hash256) -> KeyValueStoreOp {
KeyValueStoreOp::PutKeyValue(
Self::db_column(),
key.as_slice().to_vec(),
self.as_store_bytes(),
)
}
}
```

원자적 배치 작업

복잡한 상태 변경을 원자적으로 수행하기 위한 배치 작업:

```rust
#[derive(Clone)]
pub enum StoreOp<'a, E: EthSpec> {
PutBlock(Hash256, Arc<SignedBeaconBlock<E>>),
```

```
PutState(Hash256, &'a BeaconState<E>),
PutBlobs(Hash256, BlobSidecarList<E>),
PutDataColumns(Hash256, DataColumnSidecarList<E>),
PutStateSummary(Hash256, HotStateSummary),
DeleteBlock(Hash256),
DeleteBlobs(Hash256),
DeleteDataColumns(Hash256, Vec<ColumnIndex>),
DeleteState(Hash256, Option<Slot>),
DeleteExecutionPayload(Hash256),
DeleteSyncCommitteeBranch(Hash256),
KeyValueOp(KeyValueStoreOp),
}

#[derive(Clone)]
pub  enum KeyValueStoreOp {
PutKeyValue(DBColumn, Vec<u8>, Vec<u8>),
DeleteKey(DBColumn, Vec<u8>),
}
```

캐싱 전략

블록 캐시

최근 접근된 블록과 블롭을 메모리에 캐시:

```
#[derive(Debug)]
struct BlockCache<E: EthSpec> {
block_cache: LruCache<Hash256, SignedBeaconBlock<E>>,
blob_cache: LruCache<Hash256, BlobSidecarList<E>>,
data_column_cache: LruCache<Hash256, HashMap<ColumnIndex, Arc<DataColumnSidecar<
}

impl<E: EthSpec> BlockCache<E> {
pub fn new(size: NonZeroUsize) -> Self {
Self {
block_cache: LruCache::new(size),
blob_cache: LruCache::new(size),
data_column_cache: LruCache::new(size),
}
}
pub fn get_block(&mut self, block_root: &Hash256) -> Option<&SignedBeaconBloc
self.block_cache.get(block_root)
}
pub fn put_block(&mut self, block_root: Hash256, block: SignedBeaconBlock<E>)
self.block_cache.put(block_root, block);
}
```

```
}
```

상태 캐시

비콘 상태의 효율적인 캐싱을 위한 전략:

```
pub struct StateCache<E: EthSpec> {
states: LruCache<Hash256, BeaconState<E>>,
state_summaries: LruCache<Hash256, HotStateSummary>,
}

impl<E: EthSpec> StateCache<E> {
pub fn put_state(
&mut self,
state_root: Hash256,
state: BeaconState<E>,
) -> PutStateOutcome {
if self.states.contains(&state_root) {
PutStateOutcome::Duplicate
} else {
self.states.put(state_root, state);
PutStateOutcome::Fresh
}
}
```

```
pub fn get_state(&mut self, state_root: &Hash256) -> Option<&BeaconState<E>>
self.states.get(state_root)
}
}
```

데이터 정리 및 압축

자동 정리 메커니즘

```
impl<E: EthSpec> HotColdDB<E, Hot, Cold> {
/// 블롭 데이터 정리 (데이터 가용성 경계 기준)
pub fn prune_blobs(&self, current_epoch: Epoch) -> Result<(), Error> {
let data_availability_boundary = current_epoch
.safe_sub(self.spec.min_epochs_for_blob_sidecars_requests)?
.safe_sub(self.config.blob_prune_margin_epochs)?;
let boundary_slot = data_availability_boundary.start_slot(E::slots_per_epoch())
// 경계 이전의 모든 블롭 삭제
self.delete_blobs_to_slot(boundary_slot)?;
Ok(())
}
/// 실행 페이로드 정리
pub fn prune_execution_payloads(&self, finalized_slot: Slot) -> Result<(), Err
let mut ops = Vec::new();
```

```
// 최종화된 슬롯 이전의 모든 실행 페이로드 삭제
for slot in self.iter_execution_payloads_to_slot(finalized_slot)? {
let block_root = self.get_block_root(slot)?;
if let Some(root) = block_root {
ops.push(StoreOp::DeleteExecutionPayload(root));
}
}
self.do_atomically(ops)?;
Ok(())
}
}
```

데이터베이스 압축

```
impl<E: EthSpec> KeyValueStore<E> for BeaconNodeBackend<E> {
fn compact(&self) -> Result<(), Error> {
// 가장 많은 변경이 있을 것으로 예상되는 컬럼들 압축
for column in [
DBColumn::BeaconState,
DBColumn::BeaconStateHotSummary,
DBColumn::BeaconBlock,
] {
self.compact_column(column)?;
```

```
}
Ok(())
}
fn  compact_column(&self, column: DBColumn) -> Result<(), Error> {
match  self {
BeaconNodeBackend::LevelDb(db) => {
// LevelDB 컬럼별 압축
db.compact_column(column)
}
BeaconNodeBackend::Redb(db) => {
// ReDB 전체 압축 (컬럼별 압축 미지원)
db.compact()
}
}
}
}
```

메트릭 및 모니터링

성능 메트릭

저장소 작업의 성능을 모니터링하기 위한 메트릭:

```
// database/leveldb_impl.rs에서의 메트릭 수집
```

```
pub fn put_bytes_with_options(
&self,
col: DBColumn,
key: &[u8],
val: &[u8],
opts: WriteOptions,
) -> Result<(), Error> {
let column_key = get_key_for_col(col, key);
// 쓰기 작업 카운터 증가
metrics::inc_counter_vec(&metrics::DISK_DB_WRITE_COUNT, &[col.into()]);
// 쓰기 바이트 수 기록
metrics::inc_counter_vec_by(
&metrics::DISK_DB_WRITE_BYTES,
&[col.into()],
val.len() as u64,
);
// 쓰기 시간 측정 시작
let timer = metrics::start_timer(&metrics::DISK_DB_WRITE_TIMES);
let result = self.db
.put(opts.into(), BytesKey::from_vec(column_key), val)
.map_err(Into::into);
// 쓰기 시간 측정 종료
metrics::stop_timer(timer);
```

```
result
}
```

캐시 효율성 모니터링

```
impl<E: EthSpec> BlockCache<E> {
pub fn cache_hit_rate(&self) -> f64 {
let hits = self.cache_hits.load(Ordering::Relaxed);
let total = self.cache_accesses.load(Ordering::Relaxed);
if total == 0 {
0.0
} else {
hits as f64 / total as f64
}
}
pub fn record_access(&self, hit: bool) {
self.cache_accesses.fetch_add(1, Ordering::Relaxed);
if hit {
self.cache_hits.fetch_add(1, Ordering::Relaxed);
}
}
}
```

실제 사용 예시

블록 저장 및 조회

```
// 블록 저장 예시
let block_root = block.canonical_root();
let store_op = StoreOp::PutBlock(block_root, Arc::new(block));
hot_cold_db.do_atomically(vec![store_op])?;

// 블록 조회 예시
let retrieved_block: Option<SignedBeaconBlock<E>> = hot_cold_db.get(&block_root
```

상태 저장 및 복원

```
// 상태 저장
let state_root = state.canonical_root();
hot_cold_db.put_state(state_root, &state)?;

// 상태 조회 (캐시 우선)
let cached_state = hot_cold_db.get_state(&state_root)?;

// 과거 상태 복원 (복원 지점에서)
let historical_state = hot_cold_db.get_state_at_slot(historical_slot)?;
```

배치 작업 실행

```
let mut batch_ops = Vec::new();

// 여러 작업을 배치로 묶기
batch_ops.push(StoreOp::PutBlock(block_root, block));
batch_ops.push(StoreOp::PutState(state_root, &state));
batch_ops.push(StoreOp::DeleteExecutionPayload(old_payload_root));

// 원자적 실행
hot_cold_db.do_atomically(batch_ops)?;
```

결론

Lighthouse의 저장소 시스템은 Ethereum 2.0의 대용량 데이터를 효율적으로 관리하기 위해 정교하게 설계된 아키텍처입니다. Hot-Cold 분할 전략, 계층적 캐싱, 압축 최적화, 그리고 유연한 백엔드 지원을 통해 높은 성능과 확장성을 제공합니다.

주요 특징들:

1. **유연한 백엔드**: LevelDB와 ReDB 지원으로 다양한 환경에 적응
2. **효율적인 저장**: Hot-Cold 분할과 압축을 통한 공간 최적화
3. **성능 최적화**: 다층 캐싱과 배치 작업으로 높은 처리량 달성
4. **데이터 무결성**: 원자적 작업과 체크섬을 통한 안정성 보장
5. **모니터링**: 상세한 메트릭을 통한 성능 추적 및 최적화

이러한 설계를 통해 Lighthouse는 대규모 검증자 네트워크에서도 안정적이고 효율적인 데이터 관리를 제공할 수 있습니다.